

AccessFlow

AI-Powered Adaptive Accessibility Companion

Hackathon Build Guide & Technical Blueprint

Overnight Hackathon Edition | ~12 hours build time

"Make any website usable through conversation."

One AI companion. Any website. Every user.

Target Users	1B+ people with disabilities worldwide
Core Tech	Chrome Extension + LLM + Voice + Vision
Demo Impact	Works on ANY website, live
Build Time	12 hours, focused execution

Table of Contents

#	Section	Description
1	Executive Summary — The Big Idea	What AccessFlow is and why it wins
2	Problem Analysis & Opportunity	Understanding the accessibility gap
3	Solution Architecture	Technical blueprint and system design
4	Tech Stack & Setup	Every tool, library, and configuration
5	Build Timeline (Hour-by-Hour)	12-hour execution plan with milestones
6	Feature 1: Conversational Navigation	Voice/text control of any webpage
7	Feature 2: Intelligent Page Simplification	AI-driven UI adaptation
8	Feature 3: Content Description & Narration	Vision-powered accessibility
9	User Profile & Personalization	Adaptive behaviour based on needs
10	Demo Strategy & Presentation	How to deliver a winning demo
11	Judging Criteria Alignment	Mapping features to scoring rubric
12	Emergency Fallbacks & Contingencies	What to cut if running behind

1. Executive Summary

AccessFlow is an AI-powered browser companion that makes *any* website usable for people with visual, motor, cognitive, or auditory impairments — through natural conversation and real-time UI adaptation.

Instead of requiring websites to be built accessibly (which most are not), AccessFlow sits as a Chrome extension side panel that acts as a personal accessibility assistant. Users speak or type what they need, and the AI reads the page, understands the layout, and either navigates on behalf of the user, transforms the UI to be more accessible, or describes content in accessible formats.

Why This Wins a Hackathon

Live Demo Impact: Open any random website the judges suggest and show it working — proves genuine adaptability, not hardcoded demos.

End-to-End Functional: The problem statement explicitly demands a working system, not a mockup. This delivers exactly that.

Genuine Multimodality: Voice input, voice output, vision analysis of pages, text interaction, UI transformation — all four modalities.

Universal Applicability: Works on any website without requiring the website to change. This is the key differentiator from existing solutions.

Clear User Story: "Meet Sarah, who has low vision and motor difficulties. Watch her book a doctor's appointment on a website that was never built for her."

2. Problem Analysis & Opportunity

The World Health Organization estimates that over 1.3 billion people — roughly 16% of the global population — live with some form of disability. Despite decades of accessibility standards (WCAG, ADA, Section 508), the vast majority of websites remain functionally inaccessible.

The Core Failures of Current Approaches

Rigid one-size-fits-all design: Most accessibility features are binary toggles (dark mode on/off, font size up/down) rather than intelligent adaptations tailored to individual needs.

Poor assistive tech compatibility: Screen readers struggle with dynamic SPAs, complex layouts, and ARIA-unlabeled elements. Users hear gibberish or miss critical content.

High cognitive load: Complex navigation hierarchies, information-dense pages, and inconsistent layouts overwhelm users with cognitive impairments.

No multimodal alternatives: If you cannot click, most websites offer zero alternative interaction methods. If you cannot see, image-heavy pages become blank walls.

Inconsistent across updates: A website might be accessible today and broken tomorrow after a redesign. Users cannot rely on any specific workflow.

Our Insight

Rather than trying to fix every website (impossible), or building better screen readers (incremental), we use an LLM as an intelligent intermediary that can *understand* any webpage and *translate* it into whatever modality the user needs. The AI does not just read the DOM — it comprehends the page's purpose, structure, and interactive elements, then adapts the experience accordingly.

3. Solution Architecture

High-Level System Design

AccessFlow operates as a three-layer system: the **Browser Layer** (Chrome Extension) captures the current page state and executes UI modifications; the **AI Layer** (FastAPI backend + Claude API) reasons about page content and user intent; and the **Interaction Layer** (voice + text + visual feedback) provides the multimodal interface.

Layer	Component	Responsibility
Browser Layer	Chrome Extension (Manifest V3)	DOM extraction, CSS injection, element highlighting, click/scroll/type execution, side panel UI
AI Layer	FastAPI Backend + Claude Sonnet API	Page understanding, intent classification, action generation, image description, page summarization
Interaction Layer	Web Speech API + React Side Panel	Voice recognition, text-to-speech, chat interface, visual feedback overlays, user profile management
Storage Layer	Chrome Local Storage	User accessibility profile, conversation history, preference persistence

Data Flow

Step 1: User speaks or types a command in the AccessFlow side panel (e.g., "Find the search bar and type 'weather forecast'").

Step 2: The Chrome Extension extracts the current page's DOM structure, interactive elements, and a screenshot.

Step 3: This context is sent to the FastAPI backend along with the user's message and accessibility profile.

Step 4: The backend calls Claude Sonnet API with a structured prompt containing the page context, user profile, and requested action.

Step 5: Claude returns a structured JSON response specifying actions: click(selector), type(selector, text), modify_css(selector, styles), speak(text), or describe(image_url).

Step 6: The Chrome Extension executes the actions on the page and provides visual/audio feedback to the user.

Step 7: The conversation continues — the user can refine, ask follow-ups, or request different adaptations.

4. Tech Stack & Setup

Component	Technology	Why This Choice
Extension	Chrome Manifest V3	Required for DOM access, side panel API, and broad user reach
Side Panel UI	React + Tailwind CSS	Fast component-based development, looks polished quickly
Backend API	FastAPI (Python)	Your strongest framework — async, fast, minimal boilerplate
AI Model	Claude Sonnet 4 API	Fast inference, excellent structured output, vision capability built in
Voice Input	Web Speech API (SpeechRecognition)	No backend needed, runs in browser, no API key, real-time streaming
Voice Output	Web Speech API (SpeechSynthesis)	Instant, no latency, works offline, multiple voice options
Vision/Images	Claude Vision (same API)	Send base64 screenshots to Claude — no separate vision model needed
Build Tool	Vite	Lightning fast builds for the React side panel
Storage	chrome.storage.local	Persists user profile, no backend DB needed

Initial Setup Commands

Run these commands to set up your project structure:

```
# Backend setup
mkdir accessflow-backend && cd accessflow-backend
python -m venv venv && source venv/bin/activate
pip install fastapi uvicorn anthropic python-dotenv pydantic

# Frontend / Extension setup
npm create vite@latest accessflow-extension -- --template react
cd accessflow-extension
npm install tailwindcss @tailwindcss/vite

# Create the manifest.json for Chrome Extension (Manifest V3)
# Create background.js (service worker)
# Create content.js (content script for DOM access)
# Create sidepanel.html (entry point for the React side panel)
```

5. Build Timeline (Hour-by-Hour)

This is your execution roadmap. Each block has a clear deliverable. If you finish a block early, move to polish — do not scope-creep into new features.

CRITICAL RULE: Get something working end-to-end by Hour 4. Everything after that is enhancement.

Hours 0-2: Scaffolding & Hello World

Goal: Chrome extension loads, side panel opens, message goes to backend and returns a response.

Task 1 — Chrome Extension skeleton (30 min): Create manifest.json with side panel permission, background.js service worker that opens the side panel on extension icon click, and a minimal sidepanel.html. Confirm the extension loads in chrome://extensions with developer mode.

Task 2 — Content script for DOM access (30 min): Create content.js that injects into all pages. Write a function getPageContext() that extracts: page title, URL, all interactive elements (buttons, links, inputs) with their text/labels, headings hierarchy, and main content areas. Return this as a structured JSON object.

Task 3 — FastAPI backend (30 min): Set up a single POST endpoint /api/chat that accepts { message, page_context, user_profile } and returns { response_text, actions[] }. For now, just echo the message back. Confirm you can curl it.

Task 4 — Wire them together (30 min): Side panel has a text input. User types a message. Extension grabs page context via content script, sends both to the FastAPI backend, displays the response. This is your end-to-end hello world.

MILESTONE: You can type 'hello' in the side panel and get a response. The backend receives page context.

Hours 2-4: Core AI Integration

Goal: Claude API connected. User can ask about the page and get intelligent answers.

Task 5 — Claude API integration (45 min): Replace the echo in your /api/chat endpoint with a real Claude Sonnet API call. Craft a system prompt that tells Claude: "You are AccessFlow, an accessibility assistant. You receive a user's message and the current webpage context. Your job is to help the user navigate, understand, and interact with the page. Respond with helpful text AND structured actions when appropriate."

Task 6 — Structured action responses (45 min): Define an action schema that Claude returns as JSON. Actions include: click(css_selector), type(css_selector, text), scroll(direction), highlight(css_selector), read_aloud(text), simplify_page(), describe_images(). Parse these in the extension and execute them on the page.

Task 7 — Basic page interaction (30 min): Implement the click and type actions in content.js. When the AI says click('.submit-btn'), the content script finds that element and clicks it. When it says type('#search', 'weather'), it focuses the input and types. Add a visual highlight (glowing border) on targeted elements so the user sees what is happening.

MILESTONE: Ask 'What is on this page?' and get an intelligent summary. Ask 'Click the login button' and watch it happen.

Hours 4-7: The Three Killer Features

Goal: All three core features working. This is the make-or-break block.

Feature 1 — Conversational Navigation (60 min)

Enhanced DOM extraction: Improve `getPageContext()` to include landmark regions (nav, main, aside, footer), form field labels, button states (disabled/enabled), link destinations, and ARIA labels where available. The richer the context, the smarter the AI.

Multi-step actions: Enable Claude to return a sequence of actions, not just one. Example: user says 'Search for flights to Tokyo' — AI responds with `[focus('#search'), type('#search', 'flights to Tokyo'), click('.search-btn')]`.

Feedback loop: After executing actions, re-extract the DOM and send a follow-up to Claude: 'I performed the actions. Here is the updated page. Did it work?' This allows the AI to self-correct.

Feature 2 — Intelligent Page Simplification (60 min)

CSS injection engine: Build a function `applySimplification(rules)` in `content.js` that takes a list of CSS modifications and applies them. Rules include: increase font size, boost contrast, enlarge click targets (min 44x44px per WCAG), increase line spacing, remove background images, linearize multi-column layouts into single column.

AI-driven simplification: Send the page context to Claude with the prompt: 'Analyze this page for accessibility barriers. Return a list of specific CSS modifications that would make it more accessible for a user with [profile]. Focus on the most impactful changes.' The AI adapts its recommendations to the specific page AND the specific user.

One-click simplify button: Add a prominent 'Simplify Page' button in the side panel that triggers the full simplification flow. Show a before/after toggle so users (and judges) can see the transformation.

Feature 3 — Content Description & Narration (60 min)

Image description via Claude Vision: When the user asks 'describe the images on this page' or hovers/taps to an image, capture the image as base64 (use `html2canvas` or fetch the `src` directly), send it to Claude Vision, and display/speak the description. This replaces missing alt text with intelligent descriptions.

Page narration: Add a 'Read Page to Me' feature that uses Claude to generate a structured summary of the page content (not raw text dump — an intelligent summary that highlights key information, navigation options, and available actions), then uses `SpeechSynthesis` to read it aloud.

Section-by-section navigation: Let users say 'read the next section' or 'what does the main content say' for progressive disclosure rather than information overload.

MILESTONE: All three features demonstrable. A user can navigate by voice, simplify a cluttered page, and hear image descriptions.

Hours 7-9: User Profile & Voice Integration

Goal: Personalized experience. Voice interaction is smooth.

Task — Onboarding flow (45 min): When a user first opens AccessFlow, present a simple onboarding screen: 'How can I help you today?' with toggles for: Vision (low vision, colour blindness, screen reader user), Motor (limited mouse use, switch user, voice-only), Cognitive (reduce complexity, simplify language, minimize distractions), Hearing (need captions, visual alerts for audio). Store this as a profile in `chrome.storage.local`. Send it with every API request so Claude tailors its behaviour.

Task — Voice input integration (30 min): Add a microphone button in the side panel. Use the Web Speech API `SpeechRecognition` to capture voice input. Display a live transcript as the user speaks. On speech end, send the transcript as the message to the backend. Add a visual indicator (pulsing mic icon) when listening.

Task — Voice output polish (15 min): For vision-impaired users (based on profile), automatically speak all AI responses using `SpeechSynthesis`. Allow users to adjust speech rate, pitch, and voice. Add a speaker icon to toggle auto-speak.

Task — Profile-based auto-adaptation (30 min): When a user with a 'vision' profile opens any page, automatically trigger a basic simplification (contrast boost, font size increase). When a motor-impaired profile is active, automatically enlarge all click targets. This happens without the user asking — the AI anticipates needs.

MILESTONE: Open extension for first time, go through onboarding, and see the page adapt automatically to your stated needs.

Hours 9-11: Polish, Edge Cases & UI Refinement

Test on 4 real websites (60 min): Pick sites that showcase different challenges: (a) A news site like BBC or CNA — text-heavy, images, navigation; (b) A government form like SingPass or ICA — complex forms, critical interactions; (c) An e-commerce site like Amazon or Lazada — product grids, search, checkout flow; (d) A complex dashboard or data-heavy site. Fix any DOM extraction issues you find on these sites.

Side panel UI polish (30 min): Clean typography, consistent spacing, smooth animations on message appear. Add accessibility to YOUR OWN interface — keyboard navigable, high contrast, proper focus management. Ironic but essential: your accessibility tool must itself be accessible.

Error handling (30 min): What happens when: the API is slow (add loading spinner + 'I am thinking...' speech), the AI cannot find an element (graceful fallback message), the page has no images to describe (tell the user), voice recognition fails (offer text fallback). Every error state should be handled gracefully.

Hours 11-12: Demo Preparation

Goal: A scripted, rehearsed, compelling 5-minute demo.

Script your demo narrative (20 min): Do NOT just show features. Tell a story. Example script: 'This is Sarah. She has low vision and limited motor control from MS. She needs to book a medical appointment online. The clinic's website was never designed with her in mind. Watch what happens when she opens AccessFlow.' Then show: voice-activated navigation, page simplification, form filling by conversation, image descriptions.

Prepare slides (20 min): Keep slides minimal: 1) Problem (one powerful stat), 2) Solution (one sentence + architecture diagram), 3) Live Demo (the main event), 4) Tech Stack (one slide), 5) Impact and future vision. The demo IS the presentation.

Rehearse twice (20 min): Time yourself. The demo should take 3-4 minutes, leaving 1-2 minutes for Q&A.; Have a backup screen recording in case of Wi-Fi issues or API failures during the live demo.

6. Feature Deep Dive: Conversational Navigation

System Prompt Design

The system prompt is the most critical piece. Here is the structure you should use for the Claude API call:

```
You are AccessFlow, an AI accessibility companion embedded in a browser extension.
```

```
CONTEXT: You receive the user's message along with structured data about the current webpage including:
```

- Page title, URL
- All interactive elements (buttons, links, inputs) with CSS selectors and visible text
- Heading structure
- Main content areas and landmarks

```
USER PROFILE: You know the user's accessibility needs (provided per request).
```

```
YOUR TASK: Help the user interact with the webpage. Respond with:
```

1. A brief, friendly text response explaining what you are doing
2. A JSON array of actions to execute on the page

```
AVAILABLE ACTIONS:
```

- {"action": "click", "selector": "css_selector", "description": "what this does"}
- {"action": "type", "selector": "css_selector", "value": "text to type"}
- {"action": "scroll", "direction": "up|down", "amount": "pixels"}
- {"action": "highlight", "selector": "css_selector"}
- {"action": "read_aloud", "text": "text to speak to user"}
- {"action": "modify_css", "selector": "css_selector", "styles": {"property": "value"}}

```
RULES:
```

- Always confirm actions before executing destructive operations (delete, submit payment)
- If you cannot find an element, say so honestly and suggest alternatives
- Keep responses concise – the user may be hearing them read aloud
- When uncertain, highlight the candidate elements and ask for clarification

DOM Extraction Strategy

The quality of your DOM extraction directly determines how well the AI can help. Do not send the raw HTML — it is too large and noisy. Instead, extract a structured summary. Here is the approach:

Interactive elements: Query all buttons, links (a tags), inputs, selects, textareas. For each, record: a unique CSS selector (prefer id, then name, then nth-child path), visible text or label, element type, current state (checked, disabled, value), and bounding box position.

Headings: Extract all h1-h6 tags to build a structural outline of the page. This helps the AI understand the page's information architecture.

Landmarks: Identify nav, main, aside, footer, header, form elements. This gives the AI spatial awareness.

Images: Collect all img tags with their src URLs and existing alt text (or mark as 'no alt text'). Flag images larger than 100x100px as likely meaningful content.

Text content: Extract the first 500 characters of the main content area for context. Do not send the full page text — it blows up the token count.

7. Feature Deep Dive: Intelligent Page Simplification

This feature is what separates AccessFlow from a chatbot wrapper. The AI does not just talk — it physically transforms the webpage in real-time. This is viscerally impressive in a demo.

CSS Modification Categories

Category	Modifications	Target Users
Contrast & Colour	Enforce WCAG AAA contrast ratios (7:1), remove low-contrast text, add visible borders to buttons, offer high-contrast and dark mode themes	Low vision, colour blindness
Typography	Increase base font to 18px+, set line-height to 1.8, use system sans-serif fonts, limit line width to 80 characters	Low vision, dyslexia, cognitive
Click Targets	Enforce minimum 44x44px touch/click targets (WCAG), add padding to small links, space out clustered buttons	Motor impairment
Layout	Linearize multi-column to single column, remove sidebars and ads, collapse navigation menus, add clear section dividers	Cognitive, low vision
Distractions	Remove animations, hide auto-playing media, remove popups/modals, mute notification banners, reduce visual noise	Cognitive, attention
Focus & Navigation	Add visible focus outlines (3px+), implement skip-to-content links, add tab index to interactive elements, highlight current focus	Motor, keyboard-only

Implementation Guidance

Build a CSS injection utility in your content script. The function should accept a list of rules and apply them non-destructively (so they can be toggled off). Use a dedicated style tag with a unique ID so you can replace it entirely when the user changes profiles or undoes simplification.

```
// In content.js
function applyAccessibilityCSS(rules) {
  let styleEl = document.getElementById('accessflow-styles');
  if (!styleEl) {
    styleEl = document.createElement('style');
    styleEl.id = 'accessflow-styles';
    document.head.appendChild(styleEl);
  }
  styleEl.textContent = rules.map(r =>
`${r.selector} { ${r.property}: ${r.value} !important; }`

```

```
).join('\n');  
}
```

8. Feature Deep Dive: Content Description & Narration

This feature addresses the fundamental problem that screen readers read DOM text linearly without understanding context. AccessFlow uses the LLM to provide *intelligent* page narration — not 'link link button image image link' but 'This is a news article about climate policy. The main story discusses recent changes to emissions regulations. There are three images: a photo of a protest, a chart showing emissions data, and a portrait of the Environment Minister.'

Image Description Pipeline

Detect images: In content.js, find all img elements above a size threshold (100x100px). Filter out known tracking pixels, icons, and decorative images (those with role='presentation' or empty alt="").

Capture image data: For each meaningful image, get the src URL. If it is a relative URL or protected by CORS, use the Chrome extension's ability to fetch cross-origin resources. Convert to base64 if needed for the Claude Vision API.

Generate descriptions: Send each image to Claude Vision with the prompt: 'Describe this image in 1-2 sentences for a visually impaired user. Focus on: what is depicted, any text visible in the image, and the emotional tone or significance in context of [page topic].'

Deliver descriptions: Inject the descriptions as tooltips on hover/focus, and/or read them aloud via SpeechSynthesis. Add a visual indicator (small icon) on described images so sighted helpers can see which images have descriptions.

Page Narration Approach

When the user says 'Read this page to me' or 'What is on this page?', do NOT just dump all text through SpeechSynthesis. Instead, send the page context to Claude with this instruction:

```
Provide an accessibility-friendly narration of this webpage.  
Structure your narration as:  
1. One-sentence page summary (what is this page for?)  
2. Available navigation options (where can the user go?)  
3. Main content summary (what is the key information?)  
4. Available actions (what can the user do here?)  
Keep each section to 2-3 sentences maximum.  
Use plain language – reading level of grade 8 or lower.
```

9. User Profile & Personalization

Onboarding Flow Design

The onboarding must be quick (under 30 seconds) and itself accessible. It should work via both click and voice.

Profile Category	Options	Automatic Adaptations
Vision	Low vision, Colour blind, Blind (screen reader)	Auto-boost contrast and font size, auto-describe images, auto-speak responses, high contrast mode
Motor	Limited mouse use, Keyboard only, Voice only, Switch user	Enlarge click targets, enable full voice control, add keyboard shortcuts, reduce required precision
Cognitive	Reduce complexity, Simplify language, Minimize distractions, Memory support	Auto-simplify page layout, plain language summaries, remove animations and popups, add progress indicators
Hearing	Hard of hearing, Deaf	Visual alerts for audio events, captions for media, text alternatives for audio-only content

Allow users to select multiple categories — real users often have overlapping needs (e.g., low vision + motor impairment). Store the profile as a JSON object in `chrome.storage.local` and send it with every API request. The AI adapts its communication style, action suggestions, and page modifications based on this profile.

10. Demo Strategy & Presentation

The Demo Script (5 minutes)

[0:00 - 0:30] The Hook

"1.3 billion people have disabilities. 97% of websites fail basic accessibility checks. What if we could fix that — not by changing every website, but by giving every user an AI companion that adapts any website to their needs?"

[0:30 - 1:00] Meet Sarah (Persona)

Introduce your fictional user. She has low vision and motor difficulties from MS. She needs to do something specific — book a doctor's appointment, fill a government form, or shop for groceries online.

[1:00 - 1:30] Onboarding Demo

Open AccessFlow for the first time. Show the onboarding — Sarah selects 'low vision' and 'limited mouse use'. The extension immediately adapts.

[1:30 - 2:30] Page Simplification

Navigate to a real, messy website. Click 'Simplify Page'. Show the before/after transformation. Point out: larger text, better contrast, bigger buttons, cleaner layout. "Sarah can now read this page."

[2:30 - 3:30] Conversational Navigation

Sarah says (via voice): 'Find the appointment booking form'. The AI identifies it, scrolls to it, highlights it. She says 'Fill in my name as Sarah Chen'. The AI types it. She says 'Select next available date'. The AI navigates the date picker.

[3:30 - 4:00] Image Description

Navigate to an image-heavy page. Ask 'Describe the images'. Show the AI generating intelligent descriptions. Read one aloud.

[4:00 - 4:30] Wild Card

Ask a judge to suggest ANY website. Open it. Show AccessFlow working on it immediately, without any prior setup. This is the wow moment.

[4:30 - 5:00] Close

One slide: tech stack, team, and vision. "AccessFlow does not ask websites to change. It gives every user the power to adapt any website to their needs."

11. Judging Criteria Alignment

Map your demo talking points to whatever criteria the judges use. Here is how AccessFlow likely scores against common hackathon judging criteria:

Criterion	How AccessFlow Delivers	Score
Functional End-to-End System	Fully working Chrome extension with live AI backend, voice I/O, and real-time page modification. Not a mockup.	Very High
Multimodal Interaction	Four modalities: text chat, voice I/O, vision (image analysis), and UI transformation. Exceeds the 'e.g. text, speech, vision, or UI adaptation' in the problem statement.	Very High
Innovation / Creativity	Using LLM as an intelligent accessibility intermediary (not just a chatbot) that reasons about page structure. The insight that you fix the user's experience, not the website.	High
Live Demo Quality	Works on ANY website including ones judges suggest. This is the ultimate proof of generalizability.	Very High
Technical Depth	Chrome Extension APIs, Claude API with structured outputs, DOM parsing, CSS injection, Web Speech API, user profiling.	High
Impact / Scalability	1.3B potential users globally. Works on any website without requiring website changes. Low barrier to adoption.	Very High

12. Emergency Fallbacks & Contingencies

Things will go wrong overnight. Here is your triage plan for when you are running behind schedule.

If You Are Behind at Hour 6

Cut This	Keep This Instead	Impact
Voice input (speech recognition)	Text chat only, but keep voice output (TTS)	Low — text input is fine for demo; TTS still shows multimodal
AI-driven page simplification	Predefined CSS presets (high contrast, large text, simplified layout) triggered by buttons	Medium — less impressive but still shows UI adaptation
User profile persistence	In-memory toggles that reset each session	Low — judges will not test persistence
Multi-step action chains	Single actions per message (click, type, scroll)	Low — single actions are sufficient for demo
Image description via Vision API	Show existing alt text with AI enhancement of nearby text context	Medium — loses a wow factor but saves significant time

If the Live Demo Fails

Have a screen recording ready. Before the presentation, record a perfect run-through of the demo. If Wi-Fi drops or the API goes down, play the video and say 'Due to network conditions, let me show you our recorded demo.'

Cache API responses. For your demo-ready websites, cache Claude's responses locally. If the API is slow during the demo, serve cached responses. The UI still works; only the AI responses are pre-recorded.

Have a local fallback. If you cannot reach the API at all, have a simplified version that uses predefined rules (no AI) for page simplification. It is less impressive but still shows a working product.

Final Reminders

Get end-to-end working by Hour 4. Everything after that is enhancement, not core functionality.

Test on real websites early and often. DOM extraction bugs are the number one time sink.

The demo wins the hackathon, not the code. Allocate at least 90 minutes for demo prep.

If a feature is 80% done and you are running out of time, move on. 80% of three features beats 100% of one.

Sleep is optional; a working demo is not. Prioritise ruthlessly.

Have fun. The best hackathon projects come from teams that are excited about what they are building.

Good luck. Go build something that matters.